gemm_register_file

Register File và AXI4-Lite slave

Port list, giao thức MMIO, thanh ghi, chấp nhận tác vụ, trạng thái, bộ đếm

Bộ tăng tốc GEMM INT8 trên Chipyard SoC

Module	`gemm_register_file` (instance `u_regfile` trong `gemm_accel_top`)
Module con	`gemm_axil_slave`, `gemm_rf_regs`, `gemm_rf_perf`
Cấu hình	Bản đầy đủ: `DESC_PTR`, `ERR_DESC_ADDR`, `DESC_DONE_CNT` có hiệu lực (`HAS_CHAIN` = 1). Với `HAS_CHAIN` = 0 các offset này đọc 0, ghi bị bỏ qua
Tuân theo	Spec hệ thống v1.0: mục 3 (REG-01 … REG-12, TSK-01 … TSK-09, bảng 3-1 … 3-3), mục 4.5 (RST-*), mục 5.1 (AXL-01 … AXL-07), mục 6 (port `s_axi_*`), mục 7 (BS-01, DSC-07, DSC-08, PP-10)
Kèm theo	`gemm_regfile_model.py`: mô hình cycle-accurate với master AXI4-Lite ngẫu nhiên và FSM giả lập

Mục lục

1 Chức năng **3**

2 Tham số **3**

3 Port list **3**

4 Vi kiến trúc **4**

4.1 `gemm_axil_slave` 4

5 Thanh ghi **5**

5.1 `gemm_rf_perf` 6

6 Timing **6**

7 Reset, hiện thực, kiểm chứng **7**

1 Chức năng

Register File là đầu mối duy nhất giữa CPU và accelerator. Nó trả lời mọi truy cập MMIO trên AXI4-Lite, lưu các thanh ghi cấu hình, quyết định chấp nhận START hay START_CHAIN, chụp cấu hình đúng lúc chấp nhận, giữ cờ trạng thái DONE/ERROR, và đếm chu kỳ hiệu năng. Nó không biết gì về các pha: Main FSM báo lại khi tác vụ xong.

Bảng 1.1: Chức năng

ID	Yêu cầu
RF-F01	Register map, quyền truy cập và giá trị reset đúng bảng 3-1 … 3-3 spec hệ thống (REG-01)
RF-F02	Chấp nhận tác vụ theo TSK-01 … TSK-03; giữ STATUS theo TSK-04 … TSK-07, REG-06
RF-F03	Bản chụp gồm 8 thanh ghi của REG-11 và `DESC_PTR`, đủ 32 bit, xuất cho FSM và giữ nguyên tới lần chấp nhận kế tiếp
RF-F04	`PERF_*` theo REG-10. Ở chế độ ping-pong tổng ba bộ đếm thành phần được phép lớn hơn `PERF_TOTAL` (PP-10)
RF-F05	Luôn trả OKAY; không bao giờ treo bus (AXL-04)

2 Tham số

Bảng 2.1: Tham số

Tham số	Mặc định	Ý nghĩa
`P`	8	Ghi vào `HW_CONFIG[7:0]`
`HAS_CHAIN`	1	`HW_CONFIG[16]`; bật các offset 0x40 … 0x48
`HAS_PINGPONG`	1	`HW_CONFIG[17]`

3 Port list

Bảng 3.1: Port của `gemm_register_file`

Port	Rộng	Chiều	Nối với	Mô tả
`clk`, `rst_n`, `i_soft_clr`	1 × 3	In	top, FSM	
AXI4-Lite slave: 17 port `s_axi_*` của bảng 6-2 spec hệ thống, nối thẳng từ top				
Register File → FSM				
`o_start_pulse`	1	Out	FSM	Xung: một tác vụ vừa được chấp nhận
`o_start_kind`	2	Out	FSM	{START_CHAIN, START} của lệnh ghi được chấp nhận
`o_soft_req`	1	Out	FSM	Xung: lệnh ghi có SOFT_RESET = 1
`o_cfg_dim`, `o_cfg_a_addr`, `o_cfg_b_addr`, `o_cfg_c_addr`	32 × 4	Out	FSM	Bản chụp `MATRIX_DIM`, `BASE_ADDR_IN`, `_WEIGHT`, `_OUT`
`o_cfg_post_ctrl`, `o_cfg_bias_addr`, `o_cfg_qm0`, `o_cfg_qshift`	32 × 4	Out	FSM	Bản chụp `POST_CTRL`, `BIAS_ADDR`, `QUANT_M0`, `QUANT_SHIFT`
`o_cfg_desc_ptr`	32	Out	FSM	Bản chụp `DESC_PTR` (0 khi `HAS_CHAIN` = 0)
FSM → Register File				
`i_soft_busy`	1	In	FSM	Đang trong chuỗi soft reset

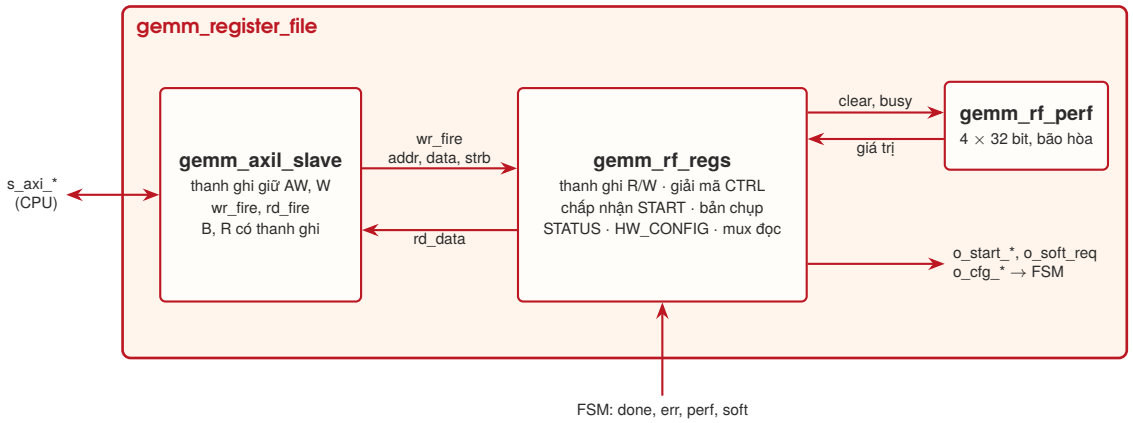
Bảng 3.1: Port của `gemm_register_file` (tiếp theo)

Port	Rộng	Chiều	Nối với	Mô tả
`i_done_pulse`	1	In	FSM	Tác vụ xong thành công
`i_err_pulse`, `i_err_code`	1, 4	In	FSM	Tác vụ xong với lỗi
`i_perf_load`, `i_perf_comp`, `i_perf_store`	1 × 3	In	FSM	Mức hoạt động của từng loại pha
`i_err_desc_addr`, `i_desc_done_cnt`	32, 32	In	FSM	Giá trị đọc của 0x44, 0x48

Bảng 3.2: Giao tiếp

ID	Yêu cầu
RF-I01	Mọi port ra (`s_axi_*` ra và `o_*`) lấy từ thanh ghi
RF-I02	`o_cfg_*` chỉ đổi ở cạnh clock chấp nhận tác vụ; ổn định suốt tác vụ (REG-11)
RF-I03	`i_done_pulse` và `i_err_pulse` chỉ đến khi BUSY = 1 và không bao giờ cùng chu kỳ (FSM-I05)

4 Vi kiến trúc



Hình 4.1: Cấu trúc bên trong `gemm_register_file`

4.1 `gemm_axil_slave`

Bảng 4.1: Giao thức AXI4-Lite

ID	Yêu cầu
AXS-01	Thanh ghi giữ `aw_v`/`aw_addr` và `w_v`/`w_data`/`w_strb`. `s_axi_awready` = ¬`aw_v` ∧ ¬`s_axi_bvalid`; `s_axi_wready` = ¬`w_v` ∧ ¬`s_axi_bvalid`. AW và W được nhận theo thứ tự bất kỳ (AXL-01)
AXS-02	`wr_fire` = `aw_v` ∧ `w_v` ∧ ¬`s_axi_bvalid`: lệnh ghi có hiệu lực ở cạnh clock cuối chu kỳ đó; cùng cạnh: `aw_v`, `w_v` ← 0, `s_axi_bvalid` ← 1, `s_axi_bresp` = 0
AXS-03	`s_axi_bvalid` giữ tới `s_axi_bready` (AXL-02). Tối đa một lệnh ghi đang xử lý (AXL-03)
AXS-04	`s_axi_arready` = ¬`s_axi_rvalid`. Ở chu kỳ bắt tay AR (`rd_fire`): `s_axi_rdata` ← giá trị đọc (bảng 5.1) tính từ trạng thái đầu chu kỳ, `s_axi_rvalid` ← 1; giữ tới `s_axi_rready`
AXS-05	Chỉ dùng bit địa chỉ [11:2] (AXL-05). Đọc và ghi cùng chu kỳ được phép; đọc trả giá trị trước lệnh ghi

5 Thanh ghi

Bảng 5.1: Hành vi từng offset (`gemm_rf_regs`)

Offset	Tên	Đọc ra	Ghi	Soft reset
0x00	CTRL	0	Lệnh, chỉ khi `strb[0]` = 1 (bảng 5.2)	—
0x04	STATUS	RFS-02, RFS-03	Bỏ qua	Về 0
0x08 … 0x24	8 thanh ghi R/W	Giá trị lưu, đủ 32 bit	Theo byte strobe (AXL-06)	Giữ
0x28	HW_CONFIG	{0x10, 6'b0, `HAS_PINGPONG`, `HAS_CHAIN`, 8'd32, 8'(P)}	Bỏ qua	—
0x2C … 0x38	`PERF_*`	Bộ đếm	Bỏ qua	Về 0
0x40	`DESC_PTR`	Giá trị lưu (`HAS_CHAIN`), ngược lại 0	Theo byte strobe (`HAS_CHAIN`), ngược lại bỏ qua	Giữ
0x44, 0x48	`ERR_DESC_ADDR`, `DESC_DONE_CNT`	`i_err_desc_addr`, `i_desc_done_cnt` (`HAS_CHAIN`), ngược lại 0	Bỏ qua	FSM xóa
Khác	dự trữ	0	Bỏ qua	—

Bảng 5.2: Lệnh ghi CTRL có `strb[0]` = 1

Bit ghi	Hành vi ở cạnh clock của `wr_fire`
SOFT_RESET = 1	`o_soft_req` ← 1 (xung ở chu kỳ sau). START, START_CHAIN bị bỏ qua (REG-05)
START ∨ START_CHAIN, và rảnh	Chấp nhận (RFS-01)
START ∨ START_CHAIN, đang bận	Bỏ qua, không đổi gì (TSK-01)
Không bit nào	Không có tác dụng

Bảng 5.3: Chấp nhận và trạng thái

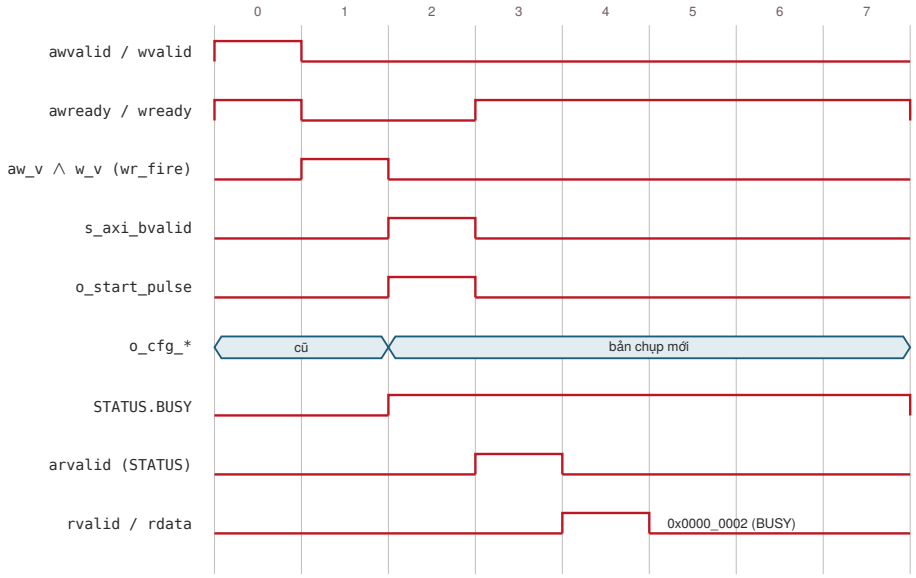
ID	Yêu cầu
RFS-01	"Rảnh" = ¬`busy_flag` ∧ ¬`i_soft_busy` ∧ ¬`o_soft_req`. Khi chấp nhận, ở cùng cạnh clock: `o_cfg_*` ← các thanh ghi R/W và `DESC_PTR`; `busy_flag` ← 1; DONE, ERROR, `ERR_CODE` ← 0; MODE ← ¬START ∧ START_CHAIN; `PERF_*` ← 0; `o_start_pulse` ← 1, `o_start_kind` ← {START_CHAIN, START}
RFS-02	STATUS = {24'b0, `ERR_CODE`, MODE, ERROR, `busy_flag`, DONE}. `i_done_pulse`: DONE ← 1, `busy_flag` ← 0. `i_err_pulse`: ERROR ← 1, `ERR_CODE` ← `i_err_code`, `busy_flag` ← 0. Như vậy BUSY về 0 đúng ở cạnh clock DONE hoặc ERROR được đặt (TSK-06)
RFS-03	Khi `i_soft_busy` = 1, STATUS đọc ra 0x0000_0002 (chỉ BUSY), nên tại mọi thời điểm tối đa một trong BUSY, DONE, ERROR bằng 1
RFS-04	`i_soft_clr`: `busy_flag`, DONE, ERROR, MODE, `ERR_CODE`, `PERF_*` ← 0 (RST-08, REG-09). Thanh ghi R/W, `DESC_PTR`, `o_cfg_*` giữ nguyên. Ưu tiên: `i_soft_clr` > chấp nhận > `i_done_pulse`/`i_err_pulse`
RFS-05	TSK-03: `o_start_pulse`, bản chụp và `busy_flag` = 1 có hiệu lực ở cùng chu kỳ `s_axi_bvalid` lên 1, nên mọi lần đọc STATUS bắt đầu sau khi CPU nhận B đều thấy tác vụ mới

5.1 gemm_rf_perf

Bảng 5.4: Bộ đếm hiệu năng (REG-10, PP-10)

ID	Yêu cầu
PRF-01	Ở mỗi chu kỳ có `busy_flag` = 1 (không phải chu kỳ chấp nhận): `PERF_TOTAL++`; `PERF_LOAD++` nếu `i_perf_load`; `PERF_COMPUTE++` nếu `i_perf_comp`; `PERF_STORE++` nếu `i_perf_store`. Mỗi bộ đếm bão hòa ở 0xFFFF_FFFF
PRF-02	Bộ đếm về 0 khi chấp nhận tác vụ, khi `i_soft_clr` và khi `rst_n`; giữ giá trị sau khi tác vụ kết thúc

6 Timing



Hình 6.1: Ghi START (AW, W cùng chu kỳ 0, `bready` = 1) rồi đọc STATUS. Lệnh ghi có hiệu lực ở cạnh clock cuối chu kỳ 1; B, xung start và BUSY cùng xuất hiện ở chu kỳ 2

Bảng 6.1: Độ trễ

Sự kiện	Chu kỳ
Bắt tay AW và W đều xong ở t → `wr_fire`	t + 1
`wr_fire` ở t → `s_axi_bvalid`, `o_start_pulse`, bản chụp, BUSY	t + 1
Bắt tay AR ở t → `s_axi_rvalid`	t + 1
`i_done_pulse` / `i_err_pulse` ở t → STATUS mới	t + 1
Thông lượng ghi liên tục	1 lệnh mỗi 3 chu kỳ khi `bready` = 1

7 Reset, hiện thực, kiểm chứng

Bảng 7.1: Reset

ID	Yêu cầu
RST-R01	`rst_n` = 0: mọi thanh ghi về giá trị reset của bảng 3-1 spec hệ thống; `aw_v`, `w_v`, `s_axi_bvalid`, `s_axi_rvalid`, `o_start_pulse`, `o_soft_req` về 0; `o_cfg_*` về 0
RST-R02	`i_soft_clr`: theo RFS-04. Giao dịch AXI4-Lite đang dở không bị ảnh hưởng

Bảng 7.2: Hiện thực

ID	Yêu cầu
IMP-01	SystemVerilog tổng hợp được, không latch. Mux đọc là `case` trên `araddr[11:2]`
IMP-02	Với `HAS_CHAIN` = 0, thanh ghi `DESC_PTR` được loại bằng `generate`

Bảng 7.3: Assertion bắt buộc (SVA)

ID	Yêu cầu
AS-01	`s_axi_bvalid` $\wedge$ ¬`s_axi_bready` $\rightarrow$ `s_axi_bvalid` ở chu kỳ sau; tương tự `rvalid` và `rdata` không đổi
AS-02	`o_start_pulse` $\rightarrow$ ¬`busy_flag` ở chu kỳ trước
AS-03	Tối đa một trong STATUS[0], STATUS[1], STATUS[2] bằng 1
AS-04	`busy_flag` $\rightarrow$ `$stable(o_cfg_*)`
AS-05	`i_done_pulse` $\vee$ `i_err_pulse` $\rightarrow$ `busy_flag`

KIỂM CHỨNG BẰNG MÔ HÌNH

`gemm_regfile_model.py`: 200 lượt $\times$ 60 lệnh ngẫu nhiên (AW và W lệch nhau 0 … 3 chu kỳ, `bready` ngẫu nhiên, byte strobe ngẫu nhiên, offset dự trữ, START/SOFT_RESET xen kẽ với FSM giả lập kết thúc tác vụ sau thời gian ngẫu nhiên): đọc lại thanh ghi R/W đúng từng byte, offset dự trữ đọc 0, STATUS không bao giờ có hai cờ cùng lúc; 300 lượt riêng kiểm TSK-03.